

第10回日本情報オリンピック JOI 2010/2011

予選 解説

2010年12月19日・情報オリンピック日本委員会

第10回日本情報オリンピック 予選

問題 1 合計時間 (Total Time)

配点 20点

競技時間 180分

採点用入力 5組

解説

この問題は、肩慣らしと JOI 予選の出題形式に慣れてもらうことを意図して出題されている。

4つの数字の合計を求め、60 で割った答えを 1 行目に出力する。同様に 60 で割った余りを 2 行目に出力する。

原文：https://www2.ioi-jp.org/joi/2010/2011-yo-prob_and_sol/2011-yo-t1/review/2011-yo-t1-review.html

問題 2 指輪 (Ring)

配点 20点

競技時間 180分

採点用入力 5組

解説

この問題は、文字列の簡単な操作によって解くことが出来る。

例えばこのような方法がある。

1. 指輪ごとに刻印されている文字列を 2 回繰り返した文字列を作る。
2. その文字列に探している文字列があるかを調べる。
3. 全ての指輪に同様の操作を行い指輪の数を数える。

以上を行うことで、前後が繋がった 10 文字から探したい文字列があるかを簡単に調べることが出来る。
たとえば、入出力例 2 では以下ようになる。

```
ZAAAAAAAAXYZAAAAAAAAXY
```

原文 : https://www2.ioi-jp.org/joi/2010/2011-yo-prob_and_sol/2011-yo-t2/review/2011-yo-t2-review.html

問題 3 タイル (Tile)

配点 20点

競技時間 180分

採点用入力 5組

解説

三色のタイルがやや複雑な形に並んでいるので、それをどう処理するかが問題である。左から a 列目、上から b 行目のタイルを (a,b) で表す。

解法 1

壁画の一番左側の辺とタイル (a,b) との距離は $a-1$ であり、一番右側の辺とタイル (a,b) との距離は $n-a$ である。また、一番上の辺とタイル (a,b) との距離は $b-1$ であり、一番下の辺とタイル (a,b) との距離は $n-b$ である。タイル (a,b) の色は「 $a-1, n-a, b-1, n-b$ の最小値を 3 で割った余り」で決まる。具体的には、余りが 0 のときは赤、余りが 1 のときは青、余りが 2 のときは黄色である。

解法 2

壁画に 2 本の対角線を引いて、次のような 4 つの領域に分割して、領域ごとに考えてもよい。

- $a \leq b, a+b \leq n+1$ をみたす領域 1
- $a \leq b, a+b > n+1$ をみたす領域 2
- $a > b, a+b \leq n+1$ をみたす領域 3
- $a > b, a+b > n+1$ をみたす領域 4

それぞれの領域ごとにタイルの色を決定することができる。領域 1 ではタイル (a,b) の色は $a-1$ を 3 で割った余りで決まる（余りが 0 のときは赤、余りが 1 のときは青、余りが 2 のときは黄色である）。また、領域 2 ではタイル (a,b) の色は $n-b$ を 3 で割った余りで決まり、同様に、領域 3 ではタイル (a,b) の色は $b-1$ を 3 で割った余りで決まり、領域 4 ではタイル (a,b) の色は $n-a$ を 3 で割った余りで決まる。

解法 3

別解として次のような方法もある。 $n \times n$ の配列を用意して、壁画のどの位置にどの色のタイルがあるかを全て調べる。多重ループを使えば、 $n \times n$ の壁画データを作成するプログラムを書くのは容易である。壁画データを作成してしまえば、どの位置のタイルが何色であるかはすぐに分かる。

この方法の方がやや直感的で、デバッグもしやすいかもしれない。この方法の欠点は計算に無駄が多く、膨大なメモリが必要なことである。 n がとても大きい場合は、メモリが足りずに $n \times n$ の配列を確保することができないかもしれない。また、仮に確保できたとしても、壁画データの作成を競技時間内に終わらせることができない可能性が高い。この問題で満点を取るには、解法 1 や解法 2 のような効率のよいアルゴリズムを設計する必要があるだろう。

原文：https://www2.ioi-jp.org/joi/2010/2011-yo-prob_and_sol/2011-yo-t3/review/2011-yo-t3-review.html

問題 4 1年生 (A First Grader)

配点 20点

競技時間 180分

採点用入力 5組

解説

数字の間にある「+」か「-」を入れる場所を「穴」と呼ぶことにする。また、書かれている n 個の数字を左から順に $a_0, \dots, a_{n-1}$ とする。

解法 1

数字が n 個並んでいるとき、穴が $n-2$ 箇所あるので、最も単純な方法として $n-2$ 箇所の穴への「+」or「-」の入れ方を（再帰呼び出しなどで）全て試してみてそれぞれ正しい数式になっているかを判定するという方法が考えられる。

この方法だと 2^{n-2} 通りの数式を調べる必要があるので $n=100$ 程度のデータを現実的な時間で解くことはできないが、いくつかのテストケースの答えを出すことができる。また、計算の途中で 0 以上 20 以下に収まらない数が現れる場合を除いていくと多少調べる量が減る。

実装は例えば下の擬似コードのようにすればよい。

```
long long answer = 0; //64bit整数
void calc(int position, int partialsum){
    //左から(position-1)番目の穴まで「+」か「-」が入っていて
    //そこまでの計算結果がpartialsum
    if(position == n-1){
        //穴を全て (n-2個) 埋めたら正しい数式になっているかチェックする
        if(partialsum == a_{n-1}) answer = answer + 1;
        //正しい数式なら答えのカウンタを1増やす
    }else{
        if(partialsum - a_{k} ≥ 0)
            calc(position + 1, partialsum - a_{k});
        if(partialsum + a_{k} ≤ 20)
            calc(position + 1, partialsum + a_{k});
    }
}
//calc(1,a_{0})で答えが求まる
```

問題 4 1年生 (A First Grader) 続き

解説 (続き)

解法 2

解法 1 は無駄が多い (※) ので、次のようにするとよい。

$f(k, B) :=$ 『左から k 個の穴に「+」 or 「-」を入れて「 a_0 から a_k までの部分の和」を B にする方法の数』
($0 \leq k \leq n-2$, $0 \leq B \leq 20$)

と置けば

- $f(k+1, B) = f(k, B-a_{k+1}) + f(k, B+a_{k+1})$ ($0 \leq B \leq 20$ でないとき $f(k, B)=0$ と考える)
- $f(0, a_0) = 1$
- $f(0, B) = 0$ ($B \neq a_0$)

より k が小さいほうから順に f の値を求めていくことができる (求める答えは $f(n-2, a_{n-1})$)。 k の取る値が $n-1$ (<100) 通りで B の取る値は 21 通りで各 $f(k, B)$ を求める操作は定数時間でできるので、この方法だと一瞬で答えを出すことができる。

(※) について

すなわち、穴への「+」 or 「-」の入れ方を全て試すアルゴリズムというのは、例えば、左から k 個の穴に「+」 or 「-」を適当に入れて「 a_0 から a_k までの部分の和 (☆)」が A になったとき、

『残っている $(n-k-2)$ 個の穴に「+」 or 「-」を入れて「 a_{k+1} から a_{n-2} までの部分の和」を $a_{n-1}-A$ にする方法の数 (＊)』

を求めることになるのであるが、左から k 個の穴をまた別の方法で埋めて (☆) が A になった時も (＊) を 1 から求めているのである。(＊) の値はいつ計算しても (=左の k 個の穴への「+」 or 「-」の入れ方によらず) 不変であることは明らかなので、これは無駄な計算である。

これを改善するには、1 度 (＊) を計算したらそのことを記憶しておいて次回以降は計算を省くようにすればよい。それには、 k と A から「(＊) を既に計算したか」「(＊) の値 (既に計算している時)」の情報を引き出せる 2 次元配列を持っておけばよい。これは実は解法 2 と本質的に同じになる。

原文 : https://www2.ioi-jp.org/joi/2010/2011-yo-prob_and_sol/2011-yo-t4/review/2011-yo-t4-review.html

問題 5 チーズ (Cheese)

配点 20点

競技時間 180分

採点用入力 5組

解説

ねずみがすべてのチーズを食べるためには、柔らかい順にチーズ工場に行かなければならない。この問題は、最短経路を N 回求めることで解くことができる。

いくつかの点（頂点と呼ぶ）と点と点を結ぶ線（辺と呼ぶ）からなるものをグラフという。グラフ G の頂点 u から頂点 v まで G の辺を通って行くとき、通らなければならない辺の個数の最小値を u と v の距離という。この問題では、区画を頂点とし、隣り合う区画に対応する頂点の間に辺を張り、そのグラフ上で距離を求めればよい。以下、グラフの頂点数を V 、辺数を E とする。

解法 1

全点間の距離を求めるアルゴリズムとしては、Floyd Warshall 法が知られている。頂点 u と頂点 v の距離を $\text{dist}(u,v)$ とおく。

```
foreach(vertex k in G){
  foreach(vertex i in G){
    foreach(vertex j in G){
      dist(i,j) := min{dist(i,j), dist(i,k) + dist(k,j)}
    }
  }
}
```

このアルゴリズムの計算量は $O(V^3)$ である。この問題で Floyd Warshall 法を用いると計算量は $O(N^3W^3)$ となり、5 データ中 2 データを解くことができる。

解法 2

ある頂点 s から他の頂点までの距離を求めるアルゴリズムとして、Bellman Ford 法が知られている。頂点 s と頂点 v の距離を $\text{dist}(v)$ とおく。

```
repeat V times:
  foreach(edge (i,j) in G){
    dist(j) := min{dist(j), dist(i) + 1}
  }
```

このアルゴリズムの計算量は $O(VE)$ である。この問題で Bellman Ford 法を用いると計算量は $O(NH^2W^2)$ となり、5 データ中 3 データを解くことができる。

問題 5 チーズ (Cheese) 続き

解説 (続き)

解法 3

キュー (queue) と呼ばれるデータ構造は、次のクエリを $O(1)$ でできる。

- 先頭の要素を参照する
- 先頭の要素を削除する
- 末尾に要素を追加する

キューを用いると、ある頂点 s から他の頂点までの距離をより高速に求める幅優先探索 (Breadth First Search) ができる。頂点 s と頂点 v の距離を $\text{dist}(v)$ とおく。

```
q := an empty queue
add s to q
while(q is not empty){
  v := the first element of q
  remove v from q
  foreach(edge (v,i) in G){
    if(dist(v) + 1 < dist(i)){
      dist(i) := dist(v) + 1
      add i to q
    }
  }
}
```

このアルゴリズムの計算量は $O(E)$ である。この問題で幅優先探索を用いると計算量は $O(NHW)$ となり、5 データ全てを解くことができる。

原文 : https://www2.ioi-jp.org/joi/2010/2011-yo-prob_and_sol/2011-yo-t5/review/2011-yo-t5-review.html

問題 6 JOI 旗 (JOI Flag)

配点 20点

競技時間 180分

採点用入力 5組

解説

この問題では、場合の数を効率よく数えることが求められている。M, N の値は小さいが、高得点を得るにはプログラムの実行時間や使用メモリの丁寧な見積もりが必要である。

J, O, I が下の図のように並んだ部分を「良い JOI」と呼ぶことにする。



「良い JOI」の配置

解法 1

最も単純な解法として、考えられるすべての「良い旗」を作りその個数を数える、という方法がある。文字が決まっていないすべての場所に対して J, O, I のどれにするかを考えたものが旗の候補であり、それぞれについて「良い旗」の条件をみたすものを数えればよい。この方法では $3^{(? \text{ の個数 })}$ 通りの候補を調べることになり、小さなデータ以外では時間がかかりすぎてしまう。

解法 1'

決まっている文字は入力の条件に従うが「良い旗」ではない旗（すなわち、J, O, I が適切に並んでいる箇所がどこにもない旗）を「悪い旗」とよぶ。「良い旗」の個数と「悪い旗」の個数を合わせると $3^{(? \text{ の個数 })}$ になるので、「悪い旗」の個数から答えは簡単に計算できる。以下では、「悪い旗」の個数を数えることにしよう。

解法 1 と同様であるが、各場所の文字を 1 つずつ順番にあてはめて試す、という方針を考える。以下のような search 関数により「悪い旗」の個数を数えることができる。

問題 6 JOI 旗 (JOI Flag) 続き

解説 (続き)

```
search ( 現在の場所 ) {
  if ( すべての場所に文字をあてはめた ) {
    answer := answer + 1
    return
  } else {
    if ( 現在の場所が ? ) {
      現在の場所に J をあてはめる
      if ( 「良い JOI」が生じなかった ) {
        search ( 次の場所 )
      }
      現在の場所に O をあてはめる
      if ( 「良い JOI」が生じなかった ) {
        search ( 次の場所 )
      }
      現在の場所に I をあてはめる
      if ( 「良い JOI」が生じなかった ) {
        search ( 次の場所 )
      }
    } else {
      (現在の場所に指定された文字をあてはめる)
      if ( 「良い JOI」が生じなかった ) {
        search ( 次の場所 )
      }
    }
  }
}
```

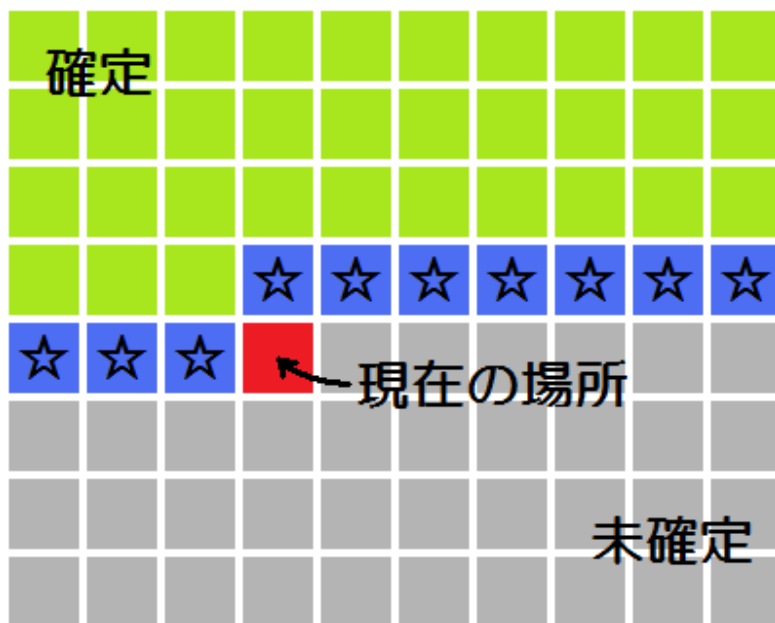
1 つでも「良い JOI」が作られた段階で「悪い旗」にはならないことがわかるので、探索を打ち切って高速化することができる。しかしこの方法でも結局最大で $3^{(? \text{ の個数 })}$ 通り、少なくとも答えの個数の旗を試すことになり、小さなデータ以外を現実的な時間で解くことはできない。

解法 2

解法 1' で生じている無駄をなくすことを考えよう。上の行から下の行へ、同じ行内では左の列から右の列へ、という順番で文字をあてはめていくものとする。下の図のような状況を考える。

問題 6 JOI 旗 (JOI Flag) 続き

解説 (続き)



現在の場所と、直前 N 個の文字による状態

現在の場所以降に文字をあてはめていく過程において、「良い JOI」が生じたかどうかを判定するときに ☆ で示した場所より上の部分にどのように文字をあてはめたかは全く関係ないことに注意する。このことにより、☆ で示した場所（現在の場所の直前 N 個）の文字が全く同じ状態にあったとき「未確定」の場所に文字をあてはめていく作業を 2 回以上行うことが無駄であるのがわかる。よって、次の値が順次計算できることになる：

（現在の場所、その直前 N 個の文字）の組に対して、現在の場所以前に「良い JOI」を作らずに文字をあてはめる方法は何通りあるか（*）。

この解法での計算時間を見積もろう。「現在の場所」は $M \times N$ 通りある。「その直前 N 個の文字」は旗の外側の場所を文字 l として考えてよいことがわかるので、 3^N 通りある。上の値はそれぞれ M, N に依らない時間で計算できるので、全体での計算時間は $3^N \times M \times N$ となる。これは M, N が 10 程度であれば十分高速に計算できる（採点用データに対しては、半分前後の部分点を獲得できる）が、 $3^{20} \times 20 \times 20 > 10^{12}$ より、最大のデータに対してはやはり時間がかかりすぎてしまう。

問題 6 JOI 旗 (JOI Flag) 続き

解説 (続き)

注意

この問題では場合の数を 100000 で割った値を答えることに注意しなければならない。さらに、考えられる「良い旗」の個数は非常に大きな値になることがあるため、「上の解法で個数を求めた後に、最後に 100000 で割って出力する」という方法を用いるとオーバーフローが発生してしまう場合がある。そこで、上述の方法で (*) の値を求めていく際に、「(*) の値を 100000 で割った余り」を代わりに求めていくようにすればよい。

解法 3

メインアイデアは解法 2 で述べたものである。☆ で示した場所について、「隣り合う 2 つの文字が "JO" となっているか否か」のみが重要であるという点に着目する。計算する値として、

(現在の場所, その直前 N 個の文字において隣り合う 2 列の文字がそれぞれ "JO" となっているか否か, 直前 1 個の文字が J であるか否か) の組に対して, 現在の場所以前に「良い JOI」を作らずに文字をあてはめる方法は何通りあるか

を考えればよい。「直前 1 個の文字が J であるか否か」は、「現在の場所」の部分に新しく "JO" ができるかどうかを調べるために使われる。

この解法での計算時間は、「その直前 n 個の文字において隣り合う 2 列の文字がそれぞれ "JO" となっているか否か」が 2^{N-1} 通りであるから, 全体で $2^N * M * N$ となる。 $2^{20} * 20 * 20 \leq 4 \times 10^8$ であり, うまくプログラムを組めば最大のデータでも数十秒以内に答えを求めることができる。

問題 6 JOI 旗 (JOI Flag) 続き

解説 (続き)

参考

解法 3 の計算時間をより丁寧に分析する。「その直前 N 個の文字において隣り合う 2 列の文字がそれぞれ "JO" となっているか否か」について、「 a 列目と $a+1$ 列目が "JO" となっていて、 $a+1$ 列目と $a+2$ 列目が "JO" になっている」ということは起こり得ない。すると、「○と×を、○が隣り合わないように $N-1$ 個並べる場合の数」を考えればよいが、これはフィボナッチ数で表されることが知られている。 $N = 20$ に対してはこれは 10946 通りしかなく、このことを利用するとさらに効率よく計算することができる。

原文 : https://www2.ioi-jp.org/joi/2010/2011-yo-prob_and_sol/2011-yo-t6/review/2011-yo-t6-review.html

出典：情報オリンピック日本委員会「第10回日本情報オリンピック JOI 2010/2011 予選 問題・採点用入力」。本 PDF は公式解説ページを問題 1 から問題 6 の順に再構成したものです。公式アーカイブでは、第5回以降の予選問題をクリエイティブ・コモンズ 表示-継承 4.0 国際 (CC BY-SA 4.0) の下で利用可能としています。

https://www2.ioi-jp.org/joi/2010/2011-yo-prob_and_sol/index.html

https://www.ioi-jp.org/problem_archive